Exercise 1 — Final Report

Real-Time Sound Classification Using Processing, FFT, OSC and Wekinator KNN Classification

Music and Multimedia Streaming Lab

Final System Configuration

Feature mode	3 — 12 reduced FFT bands
Wekinator inputs	12
Wekinator outputs	1
Output type	Classification
Classifier	K-Nearest Neighbours, $k = 3$
Class 1	`eeeeeeeeeeeee` → red canvas
Class 2	`uuuuuuuuuuuuu` → blue canvas
Total training examples	1081 (560 class 1 / 521 class 2)

OSC Communication at a Glance

Direction	OSC address	Port
Input sketch → Wekinator	`/wek/inputs`	`6448`
Wekinator → Output sketch	`/wek/outputs`	`12000`

Compiled with LaTeX on Overleaf

Contents

1 Purpose of the Exercise **1**

2 High-Level System Overview **1**

3 Main Components of the Project **2**
3.1 Input Processing Sketch . . . 2
3.2 Wekinator . . . 2
3.3 Output Processing Sketch . . . 2

4 Current Implemented Experiment **2**

5 Processing Input Sketch Configuration **3**
5.1 Audio Configuration . . . 3
5.2 Feature Mode . . . 3
5.3 OSC Input Message Sent to Wekinator . . . 4

6 FFT Feature Extraction **4**
6.1 What FFT Does . . . 4
6.2 Why FFT Is Useful for This Project . . . 4
6.3 Downsampling from 512 to 12 Bands . . . 5

7 Wekinator Configuration **5**
7.1 Input Settings . . . 5
7.2 Output Settings . . . 5
7.3 Why There Is 1 Output but 2 Classes . . . 5
7.4 Class Labels . . . 6

8 K-Nearest Neighbors Classifier **6**
8.1 What KNN Does . . . 6
8.2 What $k = 3$ Means . . . 6

9 Training Data **6**

10 Training Procedure **6**

11 Testing Procedure **7**

12 Training Stage vs Testing Stage **8**

13 Full System Requirements Checklist **8**

14 Project Deliverables **8**
14.1 Input Processing Sketch . . . 8
14.2 Output Processing Sketch . . . 8
14.3 Wekinator Project . . . 9
14.4 Final Report . . . 9

15 Discussion: Feature Dimensionality **9**
15.1 Full 512-Band FFT . . . 9
15.2 64-Band Reduced FFT . . . 9
15.3 12-Band Reduced FFT (current experiment) . . . 9
15.4 Mean and Variance . . . 9

16 Discussion: Spectral Features vs Statistical Features **9**
16.1 Spectral Features 9
16.2 Statistical Features 10

17 Possible Sources of Error **10**

18 Debugging Checklist **10**
18.1 Wekinator Does Not Receive Inputs 10
18.2 Wekinator Does Not Classify Correctly 10
18.3 Output Sketch Does Not Change Color 11

19 Important Lessons Learned **11**
19.1 Sound Must Be Converted into Features 11
19.2 Feature Choice Affects Classification 11
19.3 Wekinator Needs Matching Input Dimensions 11
19.4 Training and Running Are Separate Steps 11
19.5 OSC Connects Independent Programs 11

20 Final Project Summary **12**

21 Final Conclusion **12**

1. Purpose of the Exercise

The purpose of this exercise is to build a complete real-time sound classification system.

The system listens to live microphone audio, extracts useful numerical features from the sound, sends those features to Wekinator, trains a machine learning classifier, and finally visualizes the predicted sound class in a second Processing sketch.

The complete project combines:

- **Processing** for audio acquisition and visualization.
- **Processing Sound Library** for microphone input and FFT analysis.
- **oscP5** for communication between Processing and Wekinator.
- **Wekinator** for machine learning classification.
- **K-Nearest Neighbors** as the classification algorithm.

The final system recognizes two vocal sound classes:

- Class 1 = `eeeeeeeeeeeee` → canvas turns **red**.
- Class 2 = `uuuuuuuuuuuuu` → canvas turns **blue**.

2. High-Level System Overview

The project is divided into three main parts:

1. **Input Processing Sketch** — captures microphone audio and sends features to Wekinator.
2. **Wekinator Project** — receives features, trains a classifier, and predicts the sound class.
3. **Output Processing Sketch** — receives Wekinator's prediction and changes the screen color.

The full data flow is:

```
Microphone sound
↓
Processing input sketch
↓
FFT feature extraction
↓
OSC message:  /wek/inputs   port 6448
↓
Wekinator
↓
KNN classifier prediction
↓
OSC message:  /wek/outputs   port 12000
↓
Processing output sketch
↓
Canvas color changes
```

The system works in **real time**: the user makes a sound and immediately sees the classification result as a visual change on screen.

3. Main Components of the Project

3.1 Input Processing Sketch

The input sketch handles sound acquisition and feature extraction.

```
microphone audio → numerical features → Wekinator
```

It performs the following steps:

1. Starts the microphone.
2. Computes an FFT with 512 frequency bands.
3. Selects the feature representation based on `featureMode`.
4. Sends the features to Wekinator using OSC.
5. Displays a real-time feature bar visualization.

OSC destination:

```
OSC address        : /wek/inputs
Destination host   : 127.0.0.1  (localhost)
Destination port   : 6448
```

3.2 Wekinator

Wekinator is the machine learning component. It receives input features from Processing, learns the relationship between features and sound classes, and predicts one of two classes:

Output value	Class
1.0	eeeeeeeeeeeee
2.0	uuuuuuuuuuuuu

Wekinator uses a **K-Nearest Neighbors** classifier with $k = 3$.

3.3 Output Processing Sketch

The output sketch handles visualization only.

```
Wekinator prediction → visual feedback
```

It listens on port `12000` for messages at `/wek/outputs` and changes the background color:

Predicted class	Canvas color
Class 1	Red `(255, 0, 0)`
Class 2	Blue `(0, 0, 255)`
Waiting / unknown	Dark gray `(80)`

4. Current Implemented Experiment

The completed and tested experiment uses the **12-band FFT feature configuration**.

```
int featureMode = 3;
```

This means:

- 512 FFT bands are computed from the live microphone signal.
- Those 512 bands are downsampled into 12 averaged frequency bands.
- The 12 values are sent to Wekinator as the input feature vector.

Completed Experiment — Full Configuration

Number of Wekinator inputs	12
Number of Wekinator outputs	1
Output type	Classification
Number of classes	2
Classifier	K-Nearest Neighbors
k	3
Class 1 sound	`eeeeeeeeeeeee`
Class 2 sound	`uuuuuuuuuuuuu`
Total training examples	1081
Class 1 examples	560
Class 2 examples	521

560 vs 521 examples is a good balance. If one class had many more examples, the model could become biased toward predicting that class more often.

5. Processing Input Sketch Configuration

5.1 Audio Configuration

```
AudioIn mic;
FFT fft;
int bands = 512;
float[] spectrum = new float[bands];
```

`AudioIn` captures live sound. `FFT` analyzes frequency content. After every call to `fft.analyze(spectrum)`, the array `spectrum[0..511]` holds the current frequency magnitudes: low frequencies at index 0, high frequencies at index 511.

5.2 Feature Mode

The input sketch supports four feature modes:

featureMode	Feature representation	Wekinator inputs
1	Full FFT	512
2	Reduced FFT	64
3	Reduced FFT	12
4	Mean + variance	2

The number of values Processing sends and the number of inputs Wekinator expects must be identical. A mismatch causes silent failures — Wekinator receives data but the dimensions are wrong.

5.3 OSC Input Message Sent to Wekinator

```
OscMessage msg = new OscMessage("/wek/inputs");
for (int i = 0; i < features.length; i++) {
    msg.add(features[i]);
}
oscP5.send(msg, wekinator);
```

In the current project, `features.length = 12`, so each OSC message carries 12 float values. The destination is:

```
wekinator = new NetAddress("127.0.0.1", 6448);
```

6. FFT Feature Extraction

6.1 What FFT Does

FFT (Fast Fourier Transform) converts a time-domain audio signal into a frequency-domain representation:

```
Before FFT: sound as a changing wave over time (time domain)
↓
After FFT: energy at each of 512 frequency bands (frequency domain)
```

Wekinator cannot directly understand raw sound waves — it needs numbers. FFT converts sound into numerical features that describe frequency content.

6.2 Why FFT Is Useful for This Project

Different sounds have different spectral patterns. The vowels `eeee` and `uuuu` produce different resonance patterns in the vocal tract, leading to different frequency shapes in the FFT spectrum. A KNN classifier can learn to distinguish these patterns from the feature vectors.

6.3 Downsampling from 512 to 12 Bands

```
512 detailed FFT bands
↓
Group neighboring frequency bands
↓
Average the values inside each group
↓
12 reduced frequency features
```

Advantages of downsampling to 12 bands:

- Wekinator receives fewer inputs — simpler model.
- Faster training and lower computational cost.
- Easier to debug and visualize.
- Still captures the broad spectral shape of the sound.

The code uses proportional indexing to ensure all 512 bins are covered:

```
int start = floor(i * bands / float(targetBands));
int end   = floor((i + 1) * bands / float(targetBands));
```

This avoids ignoring remainder bins when 512 is not evenly divisible by the target (e.g., 512/12 has remainder 8).

7. Wekinator Configuration

7.1 Input Settings

```
Listening port      : 6448
Input OSC message   : /wek/inputs
Number of inputs    : 12
```

7.2 Output Settings

```
Output OSC message  : /wek/outputs
Output host         : localhost
Output port         : 12000
Number of outputs   : 1
Output type         : Classification
Number of classes   : 2
```

7.3 Why There Is 1 Output but 2 Classes

Wekinator sends one number representing the predicted class label. That single float encodes which class was recognized:

Wekinator output	Meaning
1.0	Class 1 — eeeeeeeeeeeee
2.0	Class 2 — uuuuuuuuuuuuu

One output is sufficient to represent a multi-class label in Wekinator's classification mode.

7.4 Class Labels

Class	Sound recorded	Visual result
1	`eeeeeeeeeeeee`	Red canvas
2	`uuuuuuuuuuuuu`	Blue canvas

8. K-Nearest Neighbors Classifier

8.1 What KNN Does

KNN (K-Nearest Neighbors) is a simple machine learning algorithm. When a new input feature vector arrives, KNN compares it to all recorded training examples and finds the k closest ones. It then predicts the class that appears most often among those neighbors.

8.2 What $k = 3$ Means

With $k = 3$, Wekinator finds the 3 nearest training examples and takes a majority vote:

```
Nearest example 1 -> Class 1
Nearest example 2 -> Class 1
Nearest example 3 -> Class 2
--
Result -> Class 1 (appears 2 out of 3 times)
```

With $k = 1$, a single noisy or unusual training example can strongly affect the prediction. With $k = 3$, the decision is based on a local consensus of multiple examples, making the classifier more stable.

9. Training Data

Category	Count
Total training examples	1081
Class 1 examples	560
Class 2 examples	521

The dataset is well balanced: 560 vs 521 examples means neither class dominates. An imbalanced dataset could bias the classifier toward the majority class, reducing accuracy for the minority class.

10. Training Procedure

1. **Run the input sketch.**
   The input sketch must run first because Wekinator receives features from Processing, not directly from the microphone. The sketch continuously sends `/wek/inputs` messages.

2. **Open Wekinator.**
   Create a project with 12 inputs, 1 output, 2 classes, and classifier output type. Set the

input port to `6448` and the output port to `12000`.

3. **Record Class 1.**
   Set the class dropdown to `1`. Say `eeeeeeeeeeeee` into the microphone while recording. These examples become the Class 1 training data.

4. **Record Class 2.**
   Change the class dropdown to `2`. Say `uuuuuuuuuuuuu` while recording. These examples become the Class 2 training data.

5. **Repeat recordings for both classes.**
   Recording multiple sessions per class helps Wekinator generalize across slight variations in pitch, loudness, and microphone position.

6. **Set KNN to $k = 3$.**
   Open the model settings, select K-Nearest Neighbor, and set the number of neighbors to `3`.

7. **Train the model.**
   Click **Train**. Wekinator builds the KNN classifier from the recorded examples. No examples need to be deleted when changing k — only re-training is needed.

8. **Run the model.**
   Click **Run**. Wekinator now classifies live input from the input sketch in real time.

11. Testing Procedure

After training, the complete system is tested with all three parts running simultaneously:

Component	State during testing
Input Processing sketch	Running
Wekinator	Running in **Run** mode
Output Processing sketch	Running

Expected results:

```
Say "eeeeeeeeeeeee"
-> Wekinator predicts Class 1
-> Output sketch canvas turns red
-> Text shows "Class 1"

Say "uuuuuuuuuuuuu"
-> Wekinator predicts Class 2
-> Output sketch canvas turns blue
-> Text shows "Class 2"
```

The test was successful. The classifier responded correctly and the output sketch changed color according to the predicted class.

12. Training Stage vs Testing Stage

Stage	Required components
Training	Input Processing sketch + Wekinator
Final testing	Input Processing sketch + Wekinator (Run mode) + Output Processing sketch

Wekinator does not access the microphone directly. It receives only the numerical feature vectors sent by the Processing input sketch. Without the input sketch running, Wekinator receives no live data and classification cannot work.

The output sketch is only useful after:

1. Examples have been recorded.
2. The model has been trained.
3. Wekinator has been switched to Run mode.

13. Full System Requirements Checklist

Requirement	Fulfilled by
Capture real-time microphone audio	`AudioIn` in input sketch
Compute FFT with 512 bands	`FFT`, `bands = 512`
Create reduced FFT features	Downsampling 512 → 12 bands
Send features to Wekinator via OSC	`/wek/inputs` on port `6448`
Use Wekinator for classification	Wekinator classifier project
Use KNN classifier	Wekinator model type: K-Nearest Neighbor
Use $k = 3$	Wekinator neighbor setting = 3
Train on two sound classes	`eeee` (class 1), `uuuu` (class 2)
Receive prediction in Processing	Output sketch listens on `/wek/outputs`
Change canvas color based on class	Red = class 1, blue = class 2

14. Project Deliverables

14.1 Input Processing Sketch

Captures microphone audio, extracts FFT features, and sends them to Wekinator.

Key elements: `AudioIn`, `FFT`, `featureMode`, OSC address `/wek/inputs`, port `6448`.

14.2 Output Processing Sketch

Receives Wekinator's predicted class and visualizes it as a canvas color change.

Key elements: OSC port `12000`, address `/wek/outputs`, red for class 1, blue for class 2.

14.3 Wekinator Project

Trained KNN classifier with 12 inputs, 1 output, 2 classes, $k = 3$, and 1081 training examples.

14.4 Final Report

This document. Explains input feature extraction, Wekinator configuration, classifier setup, output visualization, training and testing procedures, results, and discussion.

15. Discussion: Feature Dimensionality

Feature dimensionality means the number of input values sent to the classifier. The exercise requires comparing four configurations:

Mode	Type	# inputs	Notes
1	Full FFT	512	Maximum detail; may need more data; sensitive to noise
2	Reduced FFT	64	Good balance between detail and simplicity
3	Reduced FFT	12	Simple, fast, sufficient for clearly different sounds
4	Mean + variance	2	Very compact; loses most spectral shape information

15.1 Full 512-Band FFT

Keeps all available FFT information. May perform well for complex sound differences, but requires more training data and may be more sensitive to noise and room conditions.

15.2 64-Band Reduced FFT

A middle-ground representation. Retains more spectral detail than 12 bands while being far smaller than the full 512-band version.

15.3 12-Band Reduced FFT (current experiment)

Simple and efficient. Preserves only the broad spectral shape. This was sufficient for distinguishing `eeee` from `uuuu`, which differ in formant structure and therefore in their broad frequency envelope.

15.4 Mean and Variance

Only two values describe the entire spectrum. Useful when sounds differ greatly in overall energy or spread, but likely insufficient for vowel pairs that require spectral shape information.

16. Discussion: Spectral Features vs Statistical Features

16.1 Spectral Features

Spectral features — FFT bands in modes 1, 2, and 3 — describe how energy is distributed across frequency regions. They are powerful for sound classification because most acoustic differences between sounds are expressed in their spectral shape.

16.2 Statistical Features

Statistical features — mean and variance in mode 4 — summarize the entire spectrum with just two numbers. They are compact and fast to compute, but a single mean and variance value cannot distinguish two sounds that have the same overall energy but different spectral distributions.

The vowels `eeee` and `uuuu` differ primarily in formant frequencies — the resonant peaks of the vocal tract. These differences are visible in the spectral shape but may be invisible to mean and variance alone. Spectral features (modes 1–3) are therefore expected to outperform mode 4 for this classification task.

17. Possible Sources of Error

Error source	Effect and mitigation
Background noise	Changes FFT features; record in a quiet environment.
Inconsistent microphone distance	Features shift; keep distance constant during training and testing.
Different pitch or loudness	Features change; aim for consistent voice during recording.
Too few training examples	Unstable classifier; 1081 examples is a good quantity for this task.
Wrong number of inputs	Mismatched Processing/Wekinator dimensions; always verify `features.length` matches Wekinator's # inputs.
Wrong OSC port	Sketch receives nothing; verify all ports are consistent.

18. Debugging Checklist

18.1 Wekinator Does Not Receive Inputs

- Input sketch is running.
- Wekinator is listening on port `6448`.
- OSC input message is `/wek/inputs`.
- Wekinator expects 12 inputs.
- Microphone is working and not muted.
- Processing Sound library is installed.

18.2 Wekinator Does Not Classify Correctly

- Model was trained after setting $k = 3$.
- Wekinator is in **Run** mode.
- Both classes were recorded with enough examples.

- Background noise is low.

18.3 Output Sketch Does Not Change Color

- Output sketch is running.
- Wekinator is in **Run** mode.
- Wekinator output port is `12000`.
- Wekinator output message is `/wek/outputs`.
- Processing console prints received class values (`println`).

19. Important Lessons Learned

19.1 Sound Must Be Converted into Features

Machine learning models do not understand raw audio waveforms. Sound must first be converted into numerical feature vectors. FFT is an effective method because it captures frequency content, which is highly informative for distinguishing speech sounds.

19.2 Feature Choice Affects Classification

The selected feature representation directly affects classifier performance. A 12-band FFT is simpler than 512 bands but still contains enough information for clearly different sounds. More complex sounds may require higher-dimensional features.

19.3 Wekinator Needs Matching Input Dimensions

The number of inputs in Wekinator must always equal the number of values sent by Processing. This is the most common configuration error and should always be verified first when debugging.

19.4 Training and Running Are Separate Steps

```
Record examples -> Train -> Run -> Test live classification
```

Changing a model setting (such as k) requires re-training before the new setting takes effect. Training examples do not need to be re-recorded.

19.5 OSC Connects Independent Programs

OSC allows Processing and Wekinator to communicate in real time over the local network. The input sketch sends features to Wekinator, and Wekinator sends predictions back to the output sketch. This architecture allows each program to focus on a single responsibility.

20. Final Project Summary

Complete Final Configuration

Input sketch feature mode	3
Input features	12 reduced FFT bands
Wekinator input port	`6448`
Wekinator input message	`/wek/inputs`
Wekinator number of inputs	12
Wekinator number of outputs	1
Wekinator classes	2
Classifier	K-Nearest Neighbors
k	3
Wekinator output message	`/wek/outputs`
Wekinator output port	`12000`
Class 1 sound	`eeeeeeeeeeeee`
Class 2 sound	`uuuuuuuuuuuuu`
Total training examples	1081 (560 / 521)
Output sketch visualization	Red (class 1) / Blue (class 2)

21. Final Conclusion

This exercise successfully demonstrates a complete interactive machine learning pipeline for real-time sound classification.

The Processing input sketch transforms live microphone audio into frequency-based features using FFT. The Wekinator project uses those features to train a KNN classifier. The Processing output sketch receives the predicted class and gives immediate visual feedback.

The system fulfills all main requirements of the exercise:

- Captures real-time audio via microphone.
- Computes 512-band FFT and reduces to 12 features.
- Sends features to Wekinator using OSC (`/wek/inputs`, port 6448).
- Uses a two-class KNN classifier with $k = 3$.
- Sends classification output back to Processing (`/wek/outputs`, port 12000).
- Changes canvas color according to the predicted sound class.

The current implementation is correct for the 12-band FFT experiment. Switching to another feature configuration requires only two changes: set `featureMode` in the input sketch to `1`, `2`, or `4`, and create a matching Wekinator project with the corresponding number of inputs (512, 64, or 2).

The full interactive pipeline in one line:

```
live sound input -> FFT feature extraction -> KNN classification -> real-time
color visualization
```